

Isotope metrics for Prometheus

Optional Prometheus exporter for `kafka-isotope`. It turns the values the interceptor and consume helpers already have in hand into Micrometer meters and serves them at `GET /metrics`, a metrics-native alternative to three of the Flink reports (`latency_1m`, `topology_1m`, `hop_distribution_1m`).

Nothing new runs in your pipeline. Every value comes from what `IsotopeProducerInterceptor` and `IsotopeContext` compute anyway, and Prometheus does the 1-minute windowing at query time with `rate()` / `increase()`.

Table of Contents

- [1.0 Install](#)
 - [2.0 Put It To Work](#)
 - [2.1 Prometheus Scrape Config](#)
 - [3.0 What It Emits](#)
 - [4.0 Why Three of Seven Reports](#)
 - [5.0 Known Gaps vs. The Flink Reports](#)
 - [6.0 How It Wires In](#)
-

1.0 Install

```
dependencies {  
    implementation 'ai.signalroom:kafka-isotope-core:0.19.0'  
    implementation 'ai.signalroom:kafka-isotope-metrics:0.19.0' // optional – only  
    for Prometheus  
}
```

2.0 Put It To Work

Start the exporter once at boot, after which the interceptor you already registered starts recording. Nothing else in your code changes.

```
import ai.signalroom.kafka.isotope.metrics.PrometheusIsotopeMetrics;  
  
PrometheusIsotopeMetrics.start(9404); // serves http://0.0.0.0:9404/metrics
```

`start` is idempotent: only the first call opens the listener.

If the port is taken (commonly a sibling stage already holding 9404 on the same host), the failure is logged at WARN and swallowed. Nothing is registered, the metric calls stay inert, and your pipeline runs on. An optional sidecar never takes the application down with it. Give each stage on a host its own port.

2.1 Prometheus Scrape Config

```
scrape_configs:
  - job_name: kafka-isotope
    static_configs:
      - targets: ['order-intake-service:9404', 'order-enrichment-service:9404']
```

3.0 What It Emits

Micrometer's Prometheus registry publishes timers in seconds, as `_count`, `_sum` and `_max` series, and counters as `_total`.

Produce side, from `IsotopeProducerInterceptor.onSend`:

Metric	Tags	Gives you
<code>isotope_hop_latency_seconds_*</code>	<code>pipeline,</code> <code>origin_service,</code> <code>this_service,</code> <code>this_topic</code>	Origin → this-hop latency (avg from <code>sum/count, max</code>). Its <code>count</code> doubles as the produce-edge record count for topology.
<code>isotope_hop_records_total</code>	<code>pipeline, this_topic,</code> <code>hop_count</code>	Hop distribution: how many records reach each topic at each depth.

Consume side, from `IsotopeContext`:

Metric	Tags	Emitted by	Gives you
<code>isotope_consume_latency_seconds_*</code>	<code>pipeline,</code> <code>origin_service,</code> <code>consumer_service,</code> <code>this_topic</code>	<code>recordConsume</code>	Origin → consume time, measured when the consume marker is written.
<code>isotope_consume_records_total</code>	<code>pipeline,</code> <code>this_topic,</code> <code>consumer_service</code>	<code>recordConsume</code>	Topic → consumer edge counts, the consume half of the bipartite topology.
<code>isotope_consume_age_seconds_*</code>	<code>pipeline,</code> <code>origin_service,</code> <code>consumer_service,</code> <code>this_topic</code>	<code>adoptFromRecord(record, service),</code> or <code>recordConsume</code> for terminal consumers	How stale a record was when consumed. Emitted exactly once per

Metric	Tags	Emitted by	Gives you
			consuming stage.

The two consume paths never double-count age. A stage that adopts the trace emits it on adoption, and `recordConsume` emits it only when nothing was adopted on that thread (a terminal consumer). Use the two-argument `adoptFromRecord(record, consumerService)` to get the adoption-side sample; the one-argument form records nothing.

For the PromQL behind each report and the Grafana dashboards, see [docs/metrics.md](#) in the companion repo.

4.0 Why Three of Seven Reports

These three are pure aggregation over dimensions with a bounded number of values: service, topic, pipeline and `hop_count` (itself capped at `Isotope.MAX_HOPS`). The interceptor already has every value in scope on each `send()`, so no stream processor is needed.

The other four (merged percentiles, coverage, bipartite topology and stuck traces) need per-`trace_id` state or have to detect an event that *didn't* happen. Prometheus can express neither, so they stay in Flink. The design reasoning is in [docs/design.md](#).

5.0 Known Gaps vs. The Flink Reports

- **No `distinct_traces`**. A counter can't deduplicate, and `trace_id` is never used as a tag, because it has unbounded cardinality and would blow up the series count.
- **No windowed `min` latency**. A Micrometer timer exposes `max` but not a per-window minimum.

6.0 How It Wires In

`kafka-isotope-core` never depends on a metrics library. Its emissions route through the `IsotopeMetrics` facade to a no-op `IsotopeMetricsSink` until `start` registers `PrometheusIsotopeMetrics` in its place, so propagation runs with zero metrics overhead until you opt in. The exporter keeps its own Prometheus registry rather than the process-global one, so the isotope meters are the only thing on its `/metrics` endpoint.